GENERALITAT VALENCIANA
Conselleria d'Educació, Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC D'EDUCACIÓ A DISTÀNCIA DE LA COMUNITAT VALENCIANA

Unitat 02 — Introducció a Java
Bloc 2

1r CFGS DAW · Programació

Autores: Sergi García, Guillermo Garrido y Alfredo Oltra

En aquesta sessió

Aprofitem l'esquelet del Bloc 1: ara el codi es documenta, rep dades i aprende a fallar

1. Comentaris: `//`, `/* */` i **Javadoc**.
2. `String[] args`: donar dades al programa **abans** que arranque.
3. El compilador i els seus errors: llegir el missatge, no temer-lo.
4. El flux de treball diari en VS Code: `src`, `bin`, dreceres i autocompletat.
5. Repàs: tu ets la JVM (altra volta), Conrad el compilador i el laboratori de tortura.

ℹ️ Necessites el JDK instal·lat i VS Code amb l'extensió de Java (Bloc 1).

Comentaris: missatges per a humans

L'ordinador **ignora completament** els comentaris: són per a persones, no per a màquines.

El codi explica **què** fa la màquina; els comentaris expliquen **per què** ho fa.

I el perquè és or pur: sis mesos després, eixe comentari t'estalviarà hores de «què estava pensant quan vaig escriure açò?».

comentaris.java

```
// D'una línia: "Ací va la màgia"
/*
  De bloc: "Si això funciona, no ho toques.
  Si no funciona, no ho toques tampoc.
  Ja cridarem algú."
*/
/** Javadoc: l'elegant, genera documentació */
```

Els tres tipus de comentaris

Tipus	Sintaxi	Ús
D'una línia	`// text`	Notes ràpides al costat del codi, llistes de tasques
De bloc	`/* text */`	Explicacions llargues o desactivar codi temporalment
Javadoc	`/** text */`	Documentació formal de classes i mètodes

ℹ️ Poden anar enmig d'una línia: `System.out.println(/* "Cuatro" */ "Cinco")` imprimix **Cinco** — el comentari s'ignora i la resta seguix viva.

✅ **El consell que t'estalviarà hores:** comenta el **per què**, no el què. `// declaro i amb valor 0` és com posar «Obro la porta» en una porta.

Comenta el per què, no el què

❌ `int temperatura = 30; // temperatura val 30`

Repetix el que el codi ja diu: no aporta res.

✅ `int temperatura = 30; // refresca per davall de 25 segons el cap`

Context i intenció que el codi no pot expressar.

ℹ️ 🧠 **Truc de memòria:** si el comentari descriu la mateixa acció que veus en el codi, esborra'l. El bon comentari respon a «per què?», mai a «què?».

Javadoc: documentació que es genera sola

Javadoc (`/** ... */`) es col·loca **just abans** d'una classe o mètode. La ferramenta `javadoc` (en el JDK) ho converteix en pàgines HTML com les oficials de Java.

SobreMi.java

```java
/**
 * Classe que representa un alumne del curs.
 * @author Sergi Garcia
 * @version 1.0
 */
public class SobreMi {
    /**
     * Punt d'entrada del programa.
     * @param args arguments de la línia d'ordres
     */
    public static void main(String[] args) {
        System.out.println("Em dic Sergi");
    }
}
```

@ `@param` — per a què serveix cada paràmetre d'entrada.

@ `@return` — què torna o calcula el mètode (si no és `void`).

@ `@author` / `@version` — qui i en quin estat.

ℹ️ Els IDEs ligen estos comentaris: en passar el cursor sobre un mètode veus tota la documentació en temps real. Per a generar-la: `javadoc SobreMi.java` .

Comprovació ràpida 📝

❓ Què imprimix este programa? — `// println("Uno");` · `println("Dos");` · `/* println("Tres"); */` · `println(/* "Cuatro" */ "Cinco");`

❓ És bon comentari `// x = 10`?

1. Imprimix **Dos** i **Cinco**: les línies comentades s'ignoren i, en l'última, el comentari intern s'elimina però `"Cinco"` seguix sent l'argument. · **2.** No: el codi ja mostra que `x` val 10. Comenta el perquè, no el què.

ℹ️ 💡 En la vida real, els Javadoc dels mètodes solen «caure» en les rúbriques. I els que documenten dormen millor.

Arguments de línia d'ordres: la butxaca d'args

Recorda la firma del `main`: `public static void main(String[] args)`. `args` és una llista amb cada paraula que escrius **després** del nom del programa en executar.

```
> java UsoDeArgumentos Java mola mucho
Has escrito 3 palabras:
Palabra 1: Java
Palabra 2: mola
Palabra 3: mucho
```

ℹ️ En l'IDE: *Run → Edit Configurations → Program arguments*. En la terminal: simplement teclegeu-les després del nom de la classe.

Com s'indexen les paraules?

Índex	Valor
`args[0]`	"Java"
`args[1]`	"mola"
`args[2]`	"mucho"
`args.length`	3 — quants n'hi ha en total

⚠️ Compte amb l'error de l'aprenent: `args[0]` és el **primer** argument. Els índexs comencen en **0**, com les plantes d'un edifici: la baixa és la 0.

ℹ️ Sense arguments, `args.length` val **0** i l'array està buit. I si demanes un índex que no existix (p. ex. `args[5]` amb 3 arguments): `ArrayIndexOutOfBoundsException`.

Per a què serveix passar arguments?



Dades d'entrada: `java Calculadora 5 3` — rep 5 i 3 sense tocar el codi.



Modes d'execució: `java App --verbose` o `--silencioso`.



Arxius: `java Convertidor entrada.txt salida.txt`.



Programes reals: `git status` o `ls -la` són exactament açò.

ℹ️ Els `args` són la via d'entrada **abans** d'executar-se. Més endavant, el `Scanner` demanarà dades **durant** l'execució.

Exemple guiat: el programa que et saluda

SaludoPersonal.java

```java
public class SaludoPersonal {
  public static void main(String[] args) {
    if (args.length > 0) {
      System.out.println(
        "Hola, " + args[0] + ". ¡Benvingut!");
    } else {
      System.out.println(
        "Hola, desconegut. Has oblidat el nom?");
    }
  }
}
```

```
> java SaludoPersonal Sergi
Hola, Sergi. ¡Benvingut!
> java SaludoPersonal
Hola, desconegut. Has oblidat el nom?
```

ℹ️ El `if` ací és un aperitiu de les estructures de control que veurem en futures unitats.

Comprovació ràpida 🎒

? Amb `java MiPrograma uno dos tres` : quant val `args.length` i què conté `args[2]` ?

? Com saludes a la primera paraula que reba el teu programa?

1. `args.length` val **3** i `args[2]` conté **"tres"** (els índexs comencen en 0). · **2.** Amb `args[0]` : `System.out.println("Hola, " + args[0]);`

Compilar ≠ executar

El compilador no t'odia: és un professor de llengua puntillós que marca cada «coma mal ficada». Aprendre a llegir els seus missatges és aprendre a programar.

Compilar · `javac`

ℹ️ Tradueix el codi a bytecode. Si hi ha errors de **sintaxi**, es queixa ací i no genera el `.class`.

Error de lògica: tot «funciona», però el resultat és incorrecte. El més perillós: ni el compilador ni el runtime t'avisen → depurador.

Executar · `java`

ℹ️ La JVM executa el bytecode. Ací poden aparéixer errors de **runtime** (p. ex. `ArrayIndexOutOfBoundsException`).

Errors de compilació: et diu la **línia exacta** i el motiu — aprén a llegir-los en lloc de temer-los.

L'error de l'aprenent: 4 errors en un programa

Calculadora.java — 4 errors

```
Public class Calculadora {
    public static void main(string[] args) {
        System.out.println("Suma: " + 5 + 3);
        System.out.println("Resta: " + (5 - 3));
    }
}
```

1. `Public` → `public`: Java és sensible a majúscules.
2. Falta `{` després de `Calculadora`.
3. `string` → `String` (és una classe: S majúscula).
4. Falta el `;` final en el primer `println`.

Com llegir un error de javac

```
> javac Calculadora.java
Calculadora.java:3: error: ';' expected
System.out.println("Suma: " + 5 + 3)
^
1 error
```

1. **Arxiu i línia**: `Calculadora.java:3` — mira la `^`.
2. **El què**: `error: ';' expected` — esperava un punt i coma.
3. **El punt exacte**: la fletxeta `^` assenyala el lloc.

ℹ️ 💡 L'error sol estar en la línia marcada, però de vegades és l'**anterior**: si falta un `{` o un `;`, javac de vegades se n'adona una línia més tard. Comença en la del `^` i, si no, puja una.

Errors típics de l'aprenent (i remei)

Error	Missatge típic	Remei
Public en lloc de public	class, interface, or enum expected	Paraules clau exactes: `public`, `String`, `System`
Falta **;**	';' expected	Tota instrucció acaba en ;
Falta **{ o }**	reached end of file while parsing	Cada { ha de tindre el seu }
string en lloc de String	cannot find symbol	String és una classe: S majúscula
Classe ≠ nom de l'arxiu	should be declared in a file named X.java	La classe public es diu igual que l'arxiu .java
system en lloc de System	cannot find symbol: variable system	System amb S; out i println en minúscula

✅ 🧠 Els noms de **classes** (String, System, Scanner...) comencen en majúscula; els de **variables i mètodes** (out, println, main), en minúscula.

El lío: el codi remenat 🧩

El cap ha deixat este codi fet un desastre. Ordena les línies perquè siga un programa vàlid que imprimisca **«La suma es: 8»**.

CalculoLioso.java (desordre)

```
System.out.println("La suma es: " + (a + b));
int a = 5;
public class CalculoLioso {
System.out.println("Calculando...");
public static void main(String[] args) {
int b = 3;
```

Pista: busca primer on comença la classe i el mètode main.

Solució: la classe és el contenidor, el main la porta d'entrada i les instruccions van dins del main, en orde.

CalculoLioso.java (solució)

```
public class CalculoLioso {
    public static void main(String[] args) {
        int a = 5;
        int b = 3;
        System.out.println("Calculando...");
        System.out.println("La suma es: " + (a + b));
    }
}
```

Comprovació ràpida 😮‍💨

? Què significa `Calculadora.java:3: error: ';' expected`?

? Per què `Public` (amb P majúscula) dona error?

? Diferència entre error de compilació i error de lògica?

1. En la línia 3 de Calculadora.java, javac esperava un ; i no el va trobar (mira la ^). · **2.** Java és sensible a les majúscules: la paraula reservada és `public`, en minúscula exacta. · **3.** El de compilació impedeix generar el .class; el de lògica compila i executa, però el resultat és incorrecte i només el troba el depurador (o el sentit comú).

Un projecte Java: src i bin

estructura del projecte

```
saludo_daw/
├── src/   ← EL TEU codi (.java) viu ací
│   └── App.java
└── bin/   ← bytecode (.class) compilat
    └── App.class
```

⚠️ No edites mai els `.class` de `bin`: es regeneren en compilar. `src` és l'única font de veritat (i l'única cosa que puges a Git).

Manual, des de la terminal

```
> javac -d bin src/App.java
# -d bin: guarda el .class en bin
> java -cp bin App
# -cp bin: la JVM busca ací les classes
```

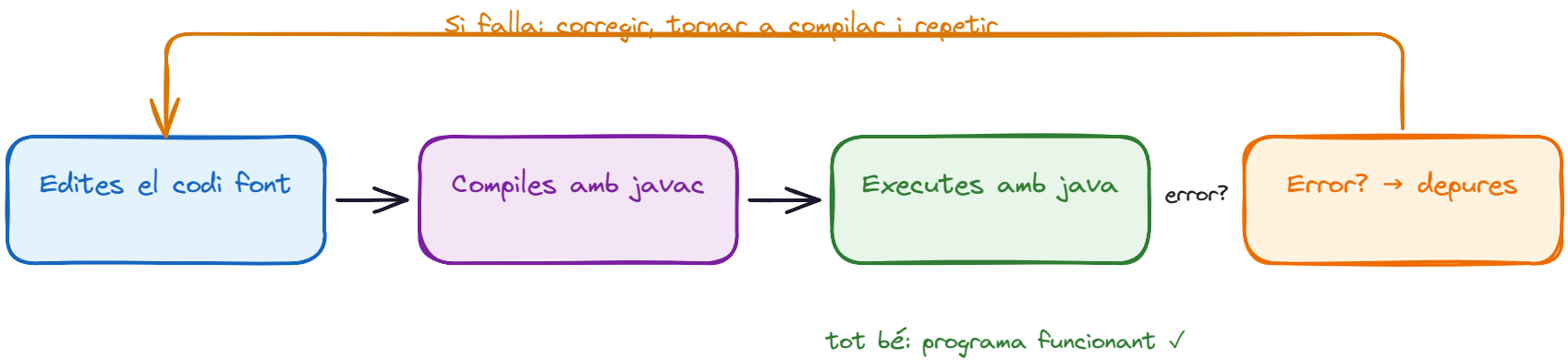
ℹ️ Amb VS Code l'eina compila **en segon pla** en guardar: ni t'enteres. Però és bo saber què passa per davall.

El cicle de treball (el teu nou bucle de vida)

1. **Editar**: escrius o canvies codi en `src`.
2. **Compilar**: VS Code ho fa automàticament en guardar (`Ctrl + S`) — ací es marquen els errors de sintaxi en roig.
3. **Executar**: `Ctrl + F5` o botó ▶ — ací apareixen els errors de runtime.
4. **Depurar**: si la lògica falla, `F5` i a treballar com detectius.

> ⚠️ Executar sense depurar (`Ctrl + F5`) i mode depuració (`F5`) **NO són el mateix**: Run ignora els breakpoints; Debug els respecta. Si poses un punt roig i executes sense depurar, el programa no es detindrà.

El cicle, dibuixat



Dreceres de VS Code que et faran paréixer un pro

Drecera	Acció
`main` + Tab (o psvm)	Escriu l'esquelet `public static void main(String[] args) {}`
`sysout` + Tab (o sout)	Escriu `System.out.println()`
Ctrl + F5 / F5	Executar / executar en mode depuració
F10 / F11	Step Over / Step Into (depurador)
Ctrl + /	Comentar o descomentar la línia seleccionada
Shift + Alt + ↓	Duplicar la línia cap avall
F12 / F2	Anar a la definició / renomenar un símbol en tot el projecte

ℹ️ 🧠 `main` i `sysout` són els dos snippets que més escriuràs: teclegeu-los i premeu Tab.

Autocompletat (IntelliSense): el teu company silenciós

1. Escrius `Syste` i VS Code t'ofereix `System` (amb la S majúscula que tant costa al principi).
2. La bombeta 💡 (o `Ctrl + .`) arregla problemes amb un clic: imports, correccions ràpides...
3. `F2` sobre una variable: la renomena en tot el projecte — açò és refactoritzar.

✅ **L'autocompletat no és trampa**: és la raó per la qual la gent usa un IDE en lloc d'un bloc de notes. El teu codi ix amb menys errors tontos perquè l'editor et corregeix mentre penses.

De zero a executar en 60 segons

1. `Ctrl + Shift + P` → *Java: Create Java Project* → *No build tools*.
2. Clic dret en `src` → New File → `HolaMundo.java`.
3. Escrius `main` + Tab, dins `sysout` + Tab...
4. `Ctrl + F5` i a mirar la Terminal.

HolaMundo.java

```java
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("¡Hola des de VS Code!");
    }
}
```

5. Breakpoint en el `println` + `F5`: observa el panell de variables.

Repàs: tracta de pensar com el codi (altra volta)

Què imprimix este programa? Tria sàviament:

Misterio.java

```
System.out.println("Café " + 1 + 2);
System.out.println("Café " + (1 + 2));
```

a) Café 3 i Café 3
b) Café 12 i Café 3
c) Café 1 2 i Café 12

L'opció b. Amb text, el `+` **concatena** d'esquerra a dreta: «Café » + 1 → «Café 1» → + 2 → «Café 12». Els parèntesis forcen la suma: «Café 3».

El clàssic que separa els qui han treballat dels qui han dormit. 😈

Repàs: qui soc? 🕵️

Endevina quin concepte soc:

1. Traduïsc el teu `.java` a bytecode. Molt puntillós amb comes i claus.
2. Execute el bytecode igual en qualsevol sistema operatiu.
3. Sóc la porta d'entrada: si canvien el meu nom, tot queda a les fosques.
4. Et deixe parar el programa i espiar les variables pas a pas.

1. El compilador (`javac`) · 2. La JVM · 3. El `main` · 4. El depurador

Repàs: CONRAD vs el món 💬

CONRAD, el compilador cascarràbies, opina sobre el clàssic dels principiants:

⚠️ «Ve un alumne i em diu: *CONRAD, no compila*. I jo pregunte: *què diu el missatge d'error?* I respon: *ah, no ho sé, no me l'he llegit*. Et done la línia exacta, el motiu i la fletxeta ^, i no ho llegeixes?»

ℹ️ «I el clàssic **Public** amb majúscula. PER QUÈ? La paraula clau és `public`, en minúscula. Fa dècades que compile i encara veig `string` en lloc de `String`...»

✅ 💡 **La lliçó:** abans de plorar sobre el teclat, llig el missatge d'error: arxiu, línia i motiu. El 90% dels errors dels principiants s'arreglen sols. **El compilador no t'odia: t'està passant les respostes de l'examen.**

Repàs: el laboratori de tortura 🧪

Copia este programa en `Tortura.java` i fes que funcione: **3 errors de compilació** i **1 error de lògica**.

Tortura.java — 3 + 1 errors

```
public class Tortura // ①
    public static void main(string[] args) {
        int a = 3;
        int b = 4;
        System.out.println("La suma es: " + a + b)// ②
        System.out.println("El producto es: " + (a * b));
    }
}
```

① falta { · ② falta ; … i queda el `string` i la suma sense parèntesis (ix «34»).

Solució: clau `{`, `String` amb S, `;` final i parèntesis per a sumar.

Tortura.java (solució)

```
public class Tortura {
    public static void main(String[] args) {
        int a = 3;
        int b = 4;
        System.out.println("La suma es: " + (a + b));
        System.out.println("El producto es: " + (a * b));
    }
}
```

Eixida: `La suma es: 7` · `El producto es: 12`

Repàs: preguntes d'entrevista de treball 💼

1. Explica'm, com si jo fora la teua iaia, la diferència entre **JDK, JRE i JVM**.

2. Què és el mètode **main** i per què té eixa firma exacta?

3. Un programa compila, però fa el que no ha de fer. **Quin és el teu procés** per a arreglar-lo?

4. Com li passes **dades a un programa** Java sense que te les demane per teclat?

ℹ️ Preguntes reals per a un programador Java júnior. Si saps respondre les quatre, la unitat està apresa.

No hi ha preguntes tontes 🤷

¿Java i JavaScript són el mateix?

Ni tan sols són del mateix planeta: Java és a JavaScript el que un gos és a un gosset calent (hot dog). Van compartir nom per pur **màrqueting** (Netscape, 1995).

¿Què és una versió LTS?

Long-Term Support: les normals reben parches 6 mesos; les LTS (8, 11, 17, 21) anys. Per això empreses i cursos trien LTS.

¿Puc escriure Java en un bloc de notes?

Pots — i és bon exercici d'humilitat —, però projectes grans amb bloc de notes és com tallar la gespa amb tisores de cuina.

¿He esborrat bin/! He perdut el treball?

No: `bin` només té bytecode regenerable. El teu treball viu en `src` — l'única font de veritat.

Resum del bloc 2 en 5 frases

1. Els comentaris són per a humans: `//`, `/* */` i `/** */` segons el que necessites.
2. Comenta el **per què**, no el què — i **Javadoc** genera la documentació automàtica.
3. `args` guarda les paraules rebudes en executar: índexs des de **0**, mida amb `args.length`.
4. Compilar i executar són moments distints amb errors distints: llig **arxiu, línia i motiu**.
5. El teu bucle diari: editar → compilar (auto) → executar → depurar, amb `src` com a única font de veritat.

// EDUCACIÓN A DISTANCIA // EDUCACIÓN PÚBLICA Y GRATUITA // EDUCACIÓN DE CALIDAD // EDUCACIÓN AUTONÓMICA //